

## Punto 3.10 - Explicit Loading

### Práctica

**Ejercicio:** Añadir métodos de carga Explicit al repositorio de órdenes del proyecto AceriaData. Crear métodos que usen `Entry().Collection().Load()`, `Entry().Reference().Load()`, `IsLoaded` y `Query()`. Crear un caso de uso que ejecute estas consultas y analice el SQL generado.

**Contexto del proyecto:** En el punto 3.9 se exploró la carga Lazy, incluyendo su configuración con proxies, el problema N+1 y sus riesgos. En este punto se explora la carga Explicit, que permite cargar entidades relacionadas de forma controlada. Esta técnica se usará en el punto 3.11 para la composición de consultas y en el Módulo 4 para las optimizaciones de rendimiento.

### Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

`cd AceriaData` → entra en la carpeta raíz del proyecto. `cd src/AceriaData.Console` → entra en la carpeta del proyecto de consola.

### Paso 2: Añadir los métodos de carga Explicit a la interfaz del repositorio

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    OrdenFabricacion? CargarPlanchasExplicitamente(int ordenId);
    OrdenFabricacion? CargarDetalleExplicitamente(int ordenId);
    OrdenFabricacion? CargarPlanchasActivasExplicitamente(int ordenId);
    OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: `using AceriaData.Application.Dtos;` → importa los DTOs. Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades. Línea 4: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres. Línea 6: `public interface IOrdenRepositorio` → declara la interfaz. Línea 8: `// ... métodos existentes ...` → comentario que indica que se mantienen los métodos anteriores. Línea 10: `OrdenFabricacion? CargarPlanchasExplicitamente(int ordenId);` → método que carga las planchas de una orden de forma explícita. Línea 11: `OrdenFabricacion? CargarDetalleExplicitamente(int ordenId);` → método que carga el detalle de una orden de forma explícita. Línea 12: `OrdenFabricacion? CargarPlanchasActivasExplicitamente(int ordenId);` → método que carga las planchas activas de una orden de forma explícita. Línea 13: `OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId);` → método que carga las planchas y el detalle de una orden de forma explícita. Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden. Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

**Error común:** si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o entidades concretas.

## Paso 3: Implementar los métodos de carga Explicit en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```
public OrdenFabricacion? CargarPlanchasExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden).Collection(o => o.Planchas).Load();
    return orden;
}

public OrdenFabricacion? CargarDetalleExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden).Reference(o => o.Detalle).Load();
    return orden;
}

public OrdenFabricacion? CargarPlanchasActivasExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden).Collection(o => o.Planchas).Query().Where(p => p.Activa).Load();
    return orden;
}

public OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    var entry = _context.Entry(orden);

    if (!entry.Collection(o => o.Planchas).IsLoaded)
    {
        entry.Collection(o => o.Planchas).Load();
    }

    if (!entry.Reference(o => o.Detalle).IsLoaded)
    {
        entry.Reference(o => o.Detalle).Load();
    }

    return orden;
}
```

Línea 1: public OrdenFabricacion? CargarPlanchasExplicitamente(int ordenId) → declara el método que carga las planchas. Línea 3: var orden = \_context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId); → carga la orden. Línea 4: if (orden is null) → comprueba si la orden existe. Línea 6: return null; → devuelve null si no existe. Línea 9: \_context.Entry(orden).Collection(o => o.Planchas).Load(); → carga la colección de planchas de forma explícita. Línea 10: return orden; → devuelve la orden con las planchas cargadas. Línea 13: public OrdenFabricacion? CargarDetalleExplicitamente(int ordenId) → declara el método que carga el detalle. Línea 15: var orden = \_context.OrdenesFabricacion.FirstOrDefault(o => o.Id

== ordenId); → carga la orden. Línea 16: if (orden is null) → comprueba si la orden existe. Línea 18: return null; → devuelve null si no existe. Línea 21: \_context.Entry(orden).Reference(o => o.Detalle).Load(); → carga la referencia al detalle de forma explícita. Línea 22: return orden; → devuelve la orden con el detalle cargado. Línea 25: public OrdenFabricacion? CargarPlanchasActivasExplicitamente(int ordenId) → declara el método que carga las planchas activas. Línea 27: var orden = \_context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId); → carga la orden. Línea 28: if (orden is null) → comprueba si la orden existe. Línea 30: return null; → devuelve null si no existe. Línea 33: \_context.Entry(orden).Collection(o => o.Planchas).Query().Where(p => p.Activa).Load(); → carga solo las planchas activas. Línea 34: return orden; → devuelve la orden con las planchas activas cargadas. Línea 37: public OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId) → declara el método que carga las planchas y el detalle. Línea 39: var orden = \_context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId); → carga la orden. Línea 40: if (orden is null) → comprueba si la orden existe. Línea 42: return null; → devuelve null si no existe. Línea 45: var entry = \_context.Entry(orden); → obtiene el EntityEntry. Línea 47: if (!entry.Collection(o => o.Planchas).IsLoaded) → comprueba si la colección ya está cargada. Línea 49: entry.Collection(o => o.Planchas).Load(); → carga la colección de planchas. Línea 52: if (!entry.Reference(o => o.Detalle).IsLoaded) → comprueba si la referencia ya está cargada. Línea 54: entry.Reference(o => o.Detalle).Load(); → carga la referencia al detalle. Línea 57: return orden; → devuelve la orden con las planchas y el detalle cargados.

**Error común:** si se llama a Load sin comprobar IsLoaded, EF Core puede ejecutar una consulta innecesaria si la propiedad ya estaba cargada por un Include previo.

## | Paso 4: Crear el caso de uso de carga Explicit

Crear el archivo src/AceriaData.Application/UseCases/CargaExplicitUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class CargaExplicitUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public CargaExplicitUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CARGA EXPLICIT ===");

        DemostrarCargaReferencia();
        DemostrarCargaColeccion();
        DemostrarCargaConFiltro();
        DemostrarCargaMultipleConIsLoaded();
    }

    private void DemostrarCargaReferencia()
    {
        Console.WriteLine("\n--- Carga Explicit de referencia (detalle) ---");

        var orden = _unidad.Ordenes.CargarDetalleExplicitamente(1);
        if (orden is not null)
        {
            Console.WriteLine($" Orden: {orden.NumeroOrden} | Cliente: {orden.Cliente}");
            var detalle = orden.Detalle == null ? "Sin detalle" :
            $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";
            Console.WriteLine($" Detalle: {detalle}");
        }
    }

    private void DemostrarCargaColeccion()
    {

```

```

        Console.WriteLine("\n--- Carga Explicit de colección (planchas) ---");

        var orden = _unidad.Ordenes.CargarPlanchasExplicitamente(1);
        if (orden is not null)
        {
            Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");
            foreach (var plancha in orden.Planchas)
            {
                Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso:
{plancha.Peso} kg");
            }
        }

        private void DemostrarCargaConFiltro()
        {
            Console.WriteLine("\n--- Carga Explicit con filtro (planchas activas) ---");

            var orden = _unidad.Ordenes.CargarPlanchasActivasExplicitamente(1);
            if (orden is not null)
            {
                Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas activas:
{orden.Planchas.Count}");
                foreach (var plancha in orden.Planchas)
                {
                    Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Activa:
{plancha.Activa}");
                }
            }

            private void DemostrarCargaMultipleConIsLoaded()
            {
                Console.WriteLine("\n--- Carga Explicit de múltiples propiedades con IsLoaded ---");

                var orden = _unidad.Ordenes.CargarPlanchasYDetalleExplicitamente(1);
                if (orden is not null)
                {
                    Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");
                    var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
                    Console.WriteLine($" Detalle: {detalle}");
                }
            }
        }
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación. Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres. Línea 5: public class CargaExplicitUseCase → declara el caso de uso. Línea 7: private readonly IUnidadDeTrabajo \_unidad; → campo de la unidad de trabajo. Línea 9: public CargaExplicitUseCase(IUnidadDeTrabajo unidad) → constructor. Línea 11: \_unidad = unidad; → asigna el parámetro al campo. Línea 14: public void Ejecutar() → declara el método principal. Línea 16: Console.WriteLine("=== CARGA EXPLICIT ==="); → muestra la cabecera. Línea 18: DemostrarCargaReferencia(); → llama al método de carga de referencia. Línea 19: DemostrarCargaColeccion(); → llama al método de carga de colección. Línea 20: DemostrarCargaConFiltro(); → llama al método de carga con filtro. Línea 21: DemostrarCargaMultipleConIsLoaded(); → llama al método de carga múltiple con IsLoaded. Línea 24: private void DemostrarCargaReferencia() → declara el método. Línea 26: Console.WriteLine("\n--- Carga Explicit de referencia (detalle) ---"); → muestra la cabecera. Línea 28: var orden = \_unidad.Ordenes.CargarDetalleExplicitamente(1); → llama al método del repositorio. Línea 29: if (orden is not null) → comprueba si la orden existe. Línea 31: Console.WriteLine(\$" Orden: {orden.NumeroOrden} | Cliente: {orden.Cliente}"); → muestra los datos de la orden. Línea 32: var detalle = orden.Detalle == null ? "Sin detalle" : \$"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C"; → comprueba si el detalle existe. Línea 33: Console.WriteLine(\$" Detalle: {detalle}"); → muestra el detalle. Línea 37: private void DemostrarCargaColeccion() → declara el método. Línea 39: Console.WriteLine("\n--- Carga Explicit de colección (planchas) ---"); → muestra la cabecera. Línea 41: var orden = \_unidad.Ordenes.CargarPlanchasExplicitamente(1); → llama al método del repositorio. Línea 42: if (orden is not null) → comprueba si la orden existe. Línea 44: Console.WriteLine(\$" Orden:

{orden.NumeroOrden} | Planchas: {orden.Planchas.Count}"); → muestra el número de planchas. Línea 45: foreach (var plancha in orden.Planchas) → itera sobre las planchas. Línea 47: Console.WriteLine(\$" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso: {plancha.Peso} kg"); → muestra los datos. Línea 51: private void DemostrarCargaConFiltro() → declara el método. Línea 53: Console.WriteLine("\n--- Carga Explicit con filtro (planchas activas) ---"); → muestra la cabecera. Línea 55: var orden = \_unidad.Ordenes.CargarPlanchasActivasExplicitamente(1); → llama al método del repositorio. Línea 56: if (orden is not null) → comprueba si la orden existe. Línea 58: Console.WriteLine(\$" Orden: {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count}"); → muestra el número de planchas activas. Línea 59: foreach (var plancha in orden.Planchas) → itera sobre las planchas. Línea 61: Console.WriteLine(\$" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Activa: {plancha.Activa}"); → muestra los datos. Línea 65: private void DemostrarCargaMultipleConIsLoaded() → declara el método. Línea 67: Console.WriteLine("\n--- Carga Explicit de múltiples propiedades con IsLoaded ---"); → muestra la cabecera. Línea 69: var orden = \_unidad.Ordenes.CargarPlanchasYDetalleExplicitamente(1); → llama al método del repositorio. Línea 70: if (orden is not null) → comprueba si la orden existe. Línea 72: Console.WriteLine(\$" Orden: {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}"); → muestra el número de planchas. Línea 73: var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica; → comprueba si el detalle existe. Línea 74: Console.WriteLine(\$" Detalle: {detalle}"); → muestra el detalle.

**Error común:** si el método del repositorio no usa IsLoaded, EF Core puede ejecutar una consulta innecesaria si la propiedad ya estaba cargada por un Include previo.

## | Paso 5: Registrar el caso de uso en el contenedor

Modificar src/AceriaData.Console/Program.cs:

```
services.AddScoped<CargaExplicitUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

**Error común:** si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

## | Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<CargaExplicitUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = \_provider.CreateScope()) → crea un ámbito. Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso. Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

**Error común:** si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

## | Paso 7: Insertar datos de prueba con planchas y detalle

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas y detalle:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    context OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = false };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%", TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    context.SaveChanges();
}

```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito. Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el DbContext. Línea 4: `context.Database.EnsureDeleted();` → elimina la base. Línea 5: `context.Database.EnsureCreated();` → crea la base. Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden. Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden. Línea 9: `context.OrdenesFabricacion.AddRange(orden1, orden2);` → registra las órdenes. Línea 10: `context.SaveChanges();` → inserta las órdenes. Línea 12: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha activa. Línea 13: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha inactiva. Línea 14: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha activa. Línea 15: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas. Línea 17: `var detalle1 = new DetalleOrden { ... };` → crea el detalle. Línea 18: `context.DetallesOrden.Add(detalle1);` → registra el detalle. Línea 20: `context.SaveChanges();` → inserta las planchas y el detalle.

**Error común:** si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

## Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observan las órdenes con detalle, con planchas, con planchas activas y con planchas y detalle. Cada sección ejecuta consultas adicionales para cargar las entidades relacionadas de forma explícita.

## Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CARGA EXPLICIT ===
```

```
--- Carga Explicit de referencia (detalle) ---
```

```
Orden: OF-2024-0001 | Cliente: Constructora del Norte
```

```
Detalle: C: 0.45%, Mn: 0.75% | 1550.5°C
```

```
--- Carga Explicit de colección (planchas) ---
```

```
Orden: OF-2024-0001 | Planchas: 2
```

```
Plancha 1 | Espesor: 10.5 | Peso: 370.5 kg
```

```
Plancha 2 | Espesor: 12.0 | Peso: 280.8 kg
```

```
--- Carga Explicit con filtro (planchas activas) ---
```

```
Orden: OF-2024-0001 | Planchas activas: 1
```

```
Plancha 1 | Espesor: 10.5 | Activa: True
```

```
--- Carga Explicit de múltiples propiedades con IsLoaded ---
```

```
Orden: OF-2024-0001 | Planchas: 2
```

```
Detalle: C: 0.45%, Mn: 0.75%
```

La primera sección muestra la orden con su detalle. La segunda sección muestra la orden con sus planchas. La tercera sección muestra la orden con sus planchas activas. La cuarta sección muestra la orden con sus planchas y su detalle.

Observaciones: en la primera sección, la carga del detalle ejecuta una consulta adicional. En la segunda sección, la carga de las planchas ejecuta una consulta adicional. En la tercera sección, la carga de las planchas activas ejecuta una consulta adicional con filtro. En la cuarta sección, la carga de las planchas y el detalle ejecuta dos consultas adicionales, pero solo si no están cargadas previamente.

Paso 10: Diagnosticar un error común

Modificar el método CargarPlanchasYDetalleExplicitamente para no comprobar IsLoaded:

```
csharp
public OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden).Collection(o => o.Planchas).Load();
    _context.Entry(orden).Reference(o => o.Detalle).Load();

    return orden;
}
```

Resultado esperado: el código funciona, pero si la colección o la referencia ya estaban cargadas por un Include previo, EF Core ejecuta una consulta innecesaria. Esto degrada el rendimiento sin aportar ningún beneficio.

Solución: usar IsLoaded para comprobar si la propiedad ya está cargada antes de llamar a Load.

```
var entry = _context.Entry(orden);

if (!entry.Collection(o => o.Planchas).IsLoaded)
{
    entry.Collection(o => o.Planchas).Load();
}

if (!entry.Reference(o => o.Detalle).IsLoaded)
{
    entry.Reference(o => o.Detalle).Load();
}
```

Resultado esperado con la solución: el código solo ejecuta consultas adicionales si la propiedad no estaba cargada previamente.

Errores comunes del ejercicio

| Error                  | Causa                                              | Solución                                         |
|------------------------|----------------------------------------------------|--------------------------------------------------|
| Consulta innecesaria   | Se llamó a Load sin comprobar IsLoaded             | Comprobar IsLoaded antes de Load                 |
| Filtro no aplicado     | Se aplicó el filtro fuera del Query                | Aplicar el filtro dentro del Query               |
| NullReferenceException | La entidad principal es <b>null</b>                | Comprobar antes de llamar a Load                 |
| Referencia circular    | Se carga una entidad que referencia a la principal | Usar DTOs o evitar cargar la referencia inversa  |
| N+1                    | Se llama a Load en un bucle                        | Usar Include en su lugar                         |
| Carga Lazy mezclada    | Se mezcla carga Explicit con carga Lazy            | Elegir una técnica y usarla de forma consistente |

Reto resuelto: Carga Explicit condicional con filtro y ordenación

Reto: Crear un método en el repositorio que cargue las planchas de una orden de forma explícita, aplicando un filtro por espesor mínimo y una ordenación por peso descendente. Añadir el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
csharp
OrdenFabricacion? CargarPlanchasFiltradasYOrdenadasExplicitamente(int ordenId, double espesorMinimo);
```

Línea 1: OrdenFabricacion? CargarPlanchasFiltradasYOrdenadasExplicitamente(int ordenId, double espesorMinimo); → declara el método.

## Paso 2: Implementar el método en OrdenRepositorio:

```
public OrdenFabricacion? CargarPlanchasFiltradasYOrdenadasExplicitamente(int ordenId, double espesorMinimo)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden)
        .Collection(o => o.Planchas)
        .Query()
        .Where(p => p.Espesor >= espesorMinimo)
        .OrderByDescending(p => p.Peso)
        .Load();

    return orden;
}
```

Línea 1: public OrdenFabricacion? CargarPlanchasFiltradasYOrdenadasExplicitamente(int ordenId, double espesorMinimo) → declara el método. Línea 3: var orden = \_context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId); → carga la orden. Línea 4: if (orden is null) → comprueba si la orden existe. Línea 6: return null; → devuelve null si no existe. Línea 9: \_context.Entry(orden) → obtiene el EntityEntry. Línea 10: .Collection(o => o.Planchas) → selecciona la colección de planchas. Línea 11: .Query() → obtiene la consulta de la colección. Línea 12: .Where(p => p.Espesor >= espesorMinimo) → filtra por espesor

mínimo. Línea 13: `.OrderByDescending(p => p.Peso)` → ordena por peso descendente. Línea 14: `.Load();` → ejecuta la consulta y carga la colección. Línea 16: `return orden;` → devuelve la orden con las planchas filtradas y ordenadas.

### Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarCargaFiltradaYOrdenada()
{
    Console.WriteLine("\n--- Carga Explicit con filtro y ordenación ---");

    var orden = _unidad.Ordenes.CargarPlanchasFiltradasYOrdenadasExplicitamente(1, 11.0);
    if (orden is not null)
    {
        Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas con espesor >= 11.0:
{orden.Planchas.Count}");
        foreach (var plancha in orden.Planchas)
        {
            Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso:
{plancha.Peso} kg");
        }
    }
}
```

Línea 1: `private void DemostrarCargaFiltradaYOrdenada()` → declara el método. Línea 3: `Console.WriteLine("\n--- Carga Explicit con filtro y ordenación ---");` → muestra la cabecera. Línea 5: `var orden = _unidad.Ordenes.CargarPlanchasFiltradasYOrdenadasExplicitamente(1, 11.0);` → llama al método del repositorio con espesor mínimo 11.0. Línea 6: `if (orden is not null)` → comprueba si la orden existe. Línea 8: `Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas con espesor >= 11.0: {orden.Planchas.Count}");` → muestra el número de planchas. Línea 9: `foreach (var plancha in orden.Planchas)` → itera sobre las planchas. Línea 11: `Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso: {plancha.Peso} kg");` → muestra los datos.

### Paso 4: Llamar al método desde Ejecutar:

```
DemostrarCargaFiltradaYOrdenada();
Paso 5: Ejecutar dotnet run y verificar que las planchas se muestran filtradas por espesor y ordenadas por peso descendente.
```

Resultado esperado: las planchas con espesor mayor o igual a 11.0 se muestran ordenadas por peso descendente.

### Analogía final

La carga Explicit en una acería es como pedirle al archivo central que te traiga las carpetas relacionadas en el momento en que las necesitas. En lugar de recibir la carpeta de la orden con las planchas y el detalle adjuntos, recibes solo la carpeta de la orden. Cuando necesitas las planchas, pides al archivo que te las traiga. Cuando necesitas el detalle, pides al archivo que te lo traiga. Cada petición es un viaje al archivo central. La carga Explicit es como un operario que decide cuándo pedir cada carpeta. Si sabe que va a necesitar las planchas, las pide. Si no las necesita, no las pide. `IsLoaded` es como comprobar si ya tienes la carpeta antes de pedirla de nuevo. Query es como pedir solo las planchas que cumplen una condición. La carga Explicit es más controlada que la carga Lazy pero requiere más código. En una acería con mucho volumen, la carga Eager es la opción más eficiente. En una aplicación donde no se sabe de antemano qué se va a necesitar, la carga Explicit es una buena alternativa. Así funciona la carga Explicit en EF Core: manual, controlada y explícita.

### Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de carga Explicit a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `CargaExplicitUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas activas e inactivas, y detalle.

Ejecutado las demostraciones de carga de referencia, carga de colección, carga con filtro y carga múltiple con `IsLoaded`.

Diagnosticado el error de no comprobar `IsLoaded`.

Creado el método `CargarPlanchasFiltradasYOrdenadasExplicitamente`.

## **Conexión con el siguiente punto**

En este punto se ha explorado la carga Explicit, incluyendo `Entry().Reference().Load()`, `Entry().Collection().Load()`, `IsLoaded` y `Query()`. Se ha comprobado que la carga Explicit es manual y controlada, y que se puede aplicar filtros y ordenaciones antes de cargar. En el siguiente punto se estudiará la composición de consultas con `IQueryable`, incluyendo la ejecución diferida y las consultas condicionales.

---